

CSE110A: Compilers

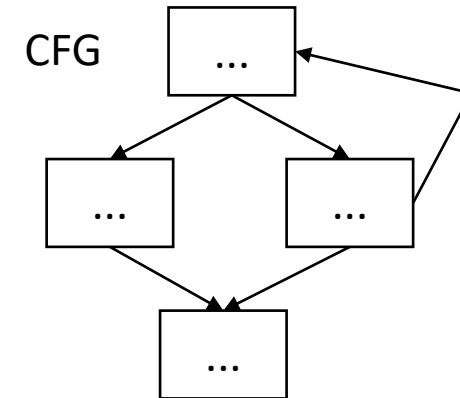
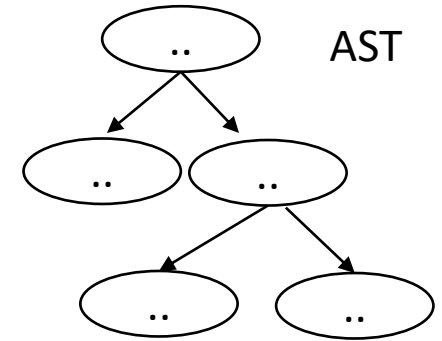
Topics: Final Review

Module 3: Intermediate Representations

- *Overview 2*
- *ASTs and Type Inferencing 8*
- *Intermediate Representations 35*
- *Scopes 48*

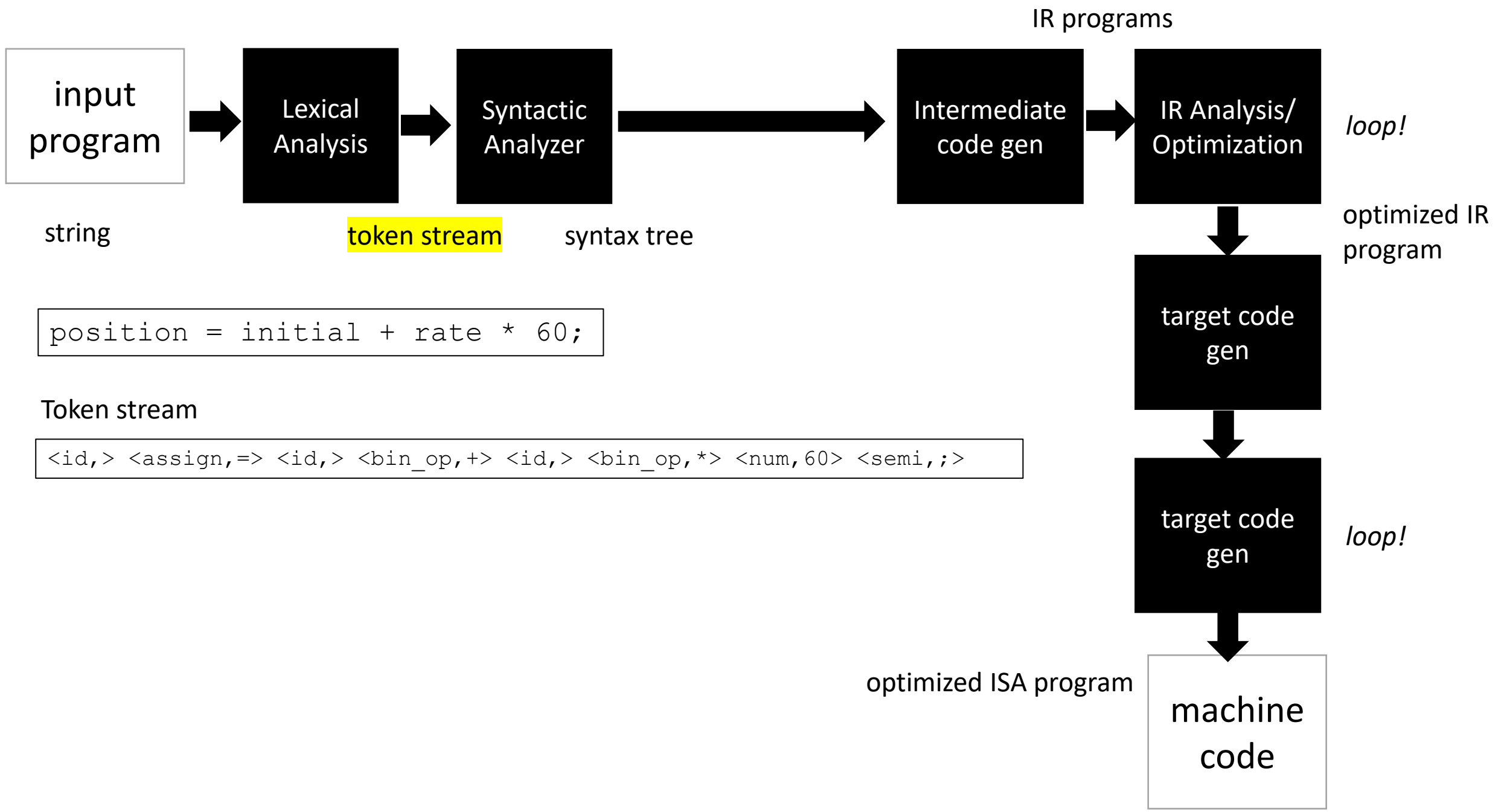
Module 4: Optimizations

- *Basic Blocks 49*
- *Control Flow Graphs 50*
- *Local Value Numbering (LVN) 51*
- *Stitching LVN 79*
- *Loop Unrolling 85*



3 address code

```
store i32 0, ptr %2
%3 = load i32, ptr %1
%4 = add nsw i32 %3, 1,
store i32 %4, ptr %1
%5 = load i32, ptr %2
```



```
position = initial + rate * 60;
```

input
program



Lexical
Analysis



Syntactic
Analyzer



Intermediate
code gen



IR Analysis/
Optimization

loop!

optimized IR
program



target code
gen



target code
gen

loop!



machine
code

string

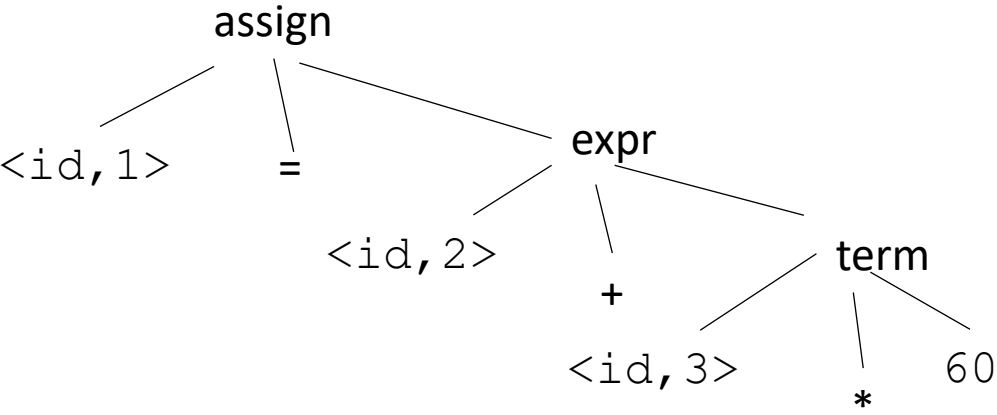
token stream

syntax tree

Token stream

```
<id,> <assign,=> <id,> <bin_op,+> <id,> <bin_op,*> <num,60> <semi,;>
```

Syntax tree



```
position = initial + rate * 60;
```

input
program



Lexical
Analysis



Syntactic
Analyzer



Intermediate
code gen



IR Analysis/
Optimization

loop!

optimized IR
program



target code
gen



target code
gen

loop!



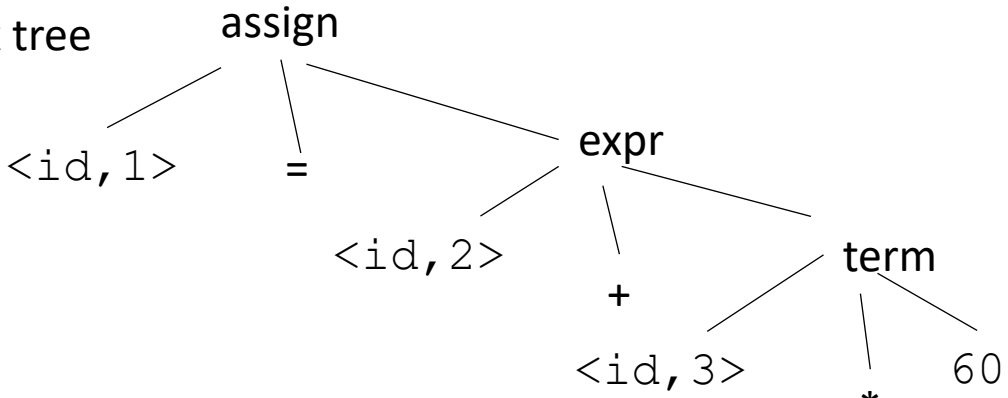
machine
code

string

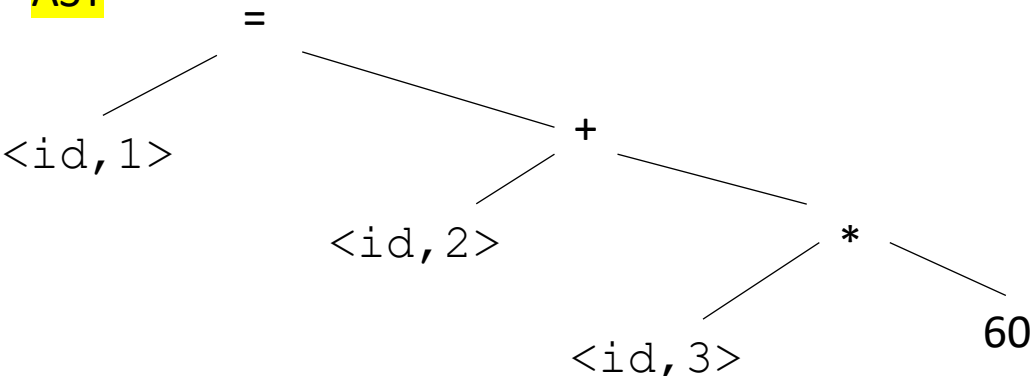
token stream

syntax tree

Syntax tree



AST



```
position = initial + rate * 60;
```

input
program

string

Lexical
Analysis

token stream

Syntactic
Analyzer

syntax tree

Intermediate
code gen

IR programs

IR Analysis/
Optimization

loop!

optimized IR
program

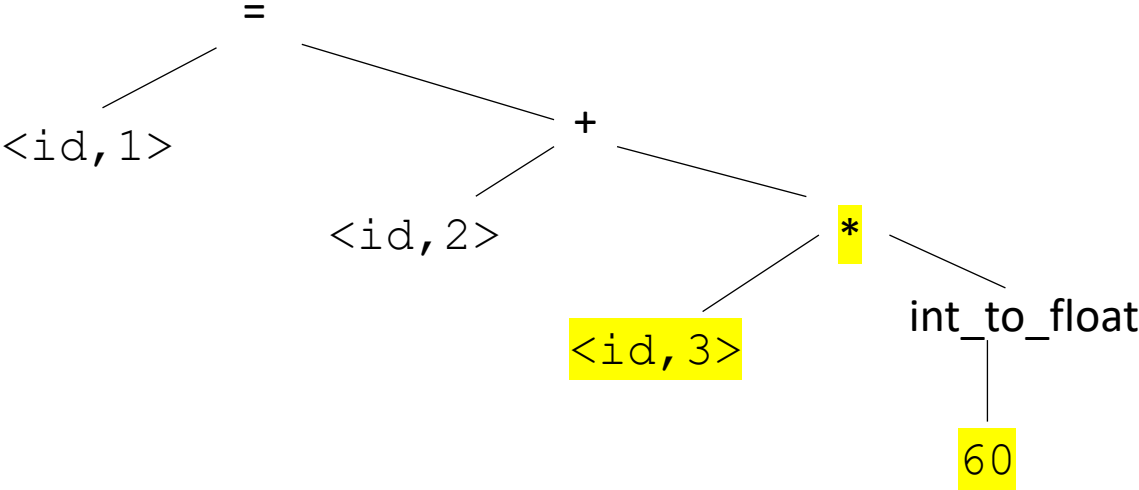
target code
gen

target code
gen

loop!

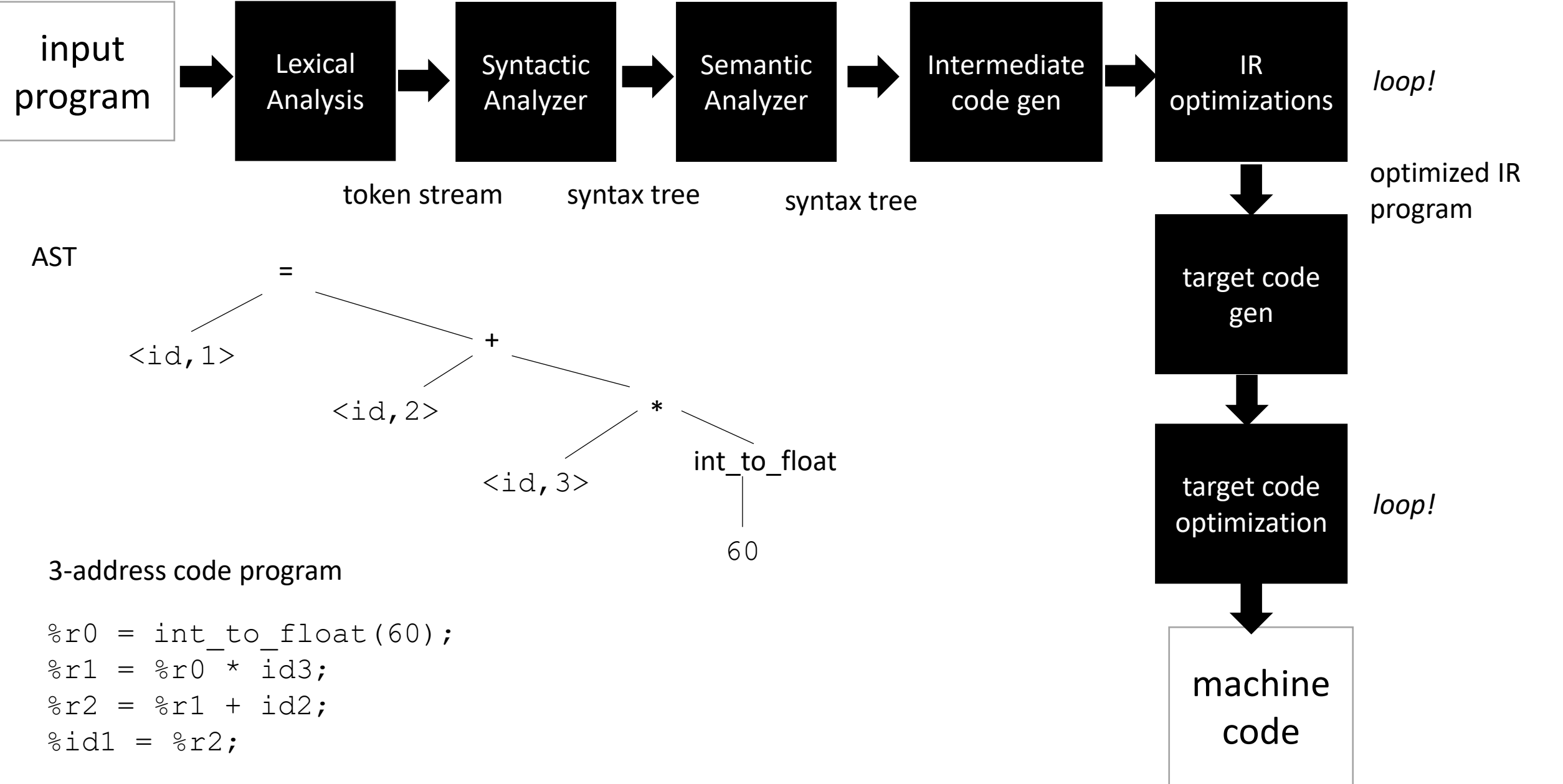
machine
code

AST



```
position = initial + rate * 60;
```

IR programs



Intermediate representations

- Several forms:
 - tree - abstract syntax tree
 - graphs - control flow graph
 - linear program - 3 address code
- Often times the program is represented as a hybrid
 - graphs where nodes are a linear program
 - linear program where expressions are ASTs
- Progression:
 - start close to a parse tree
 - move closer to an ISA

Abstract Syntax Trees

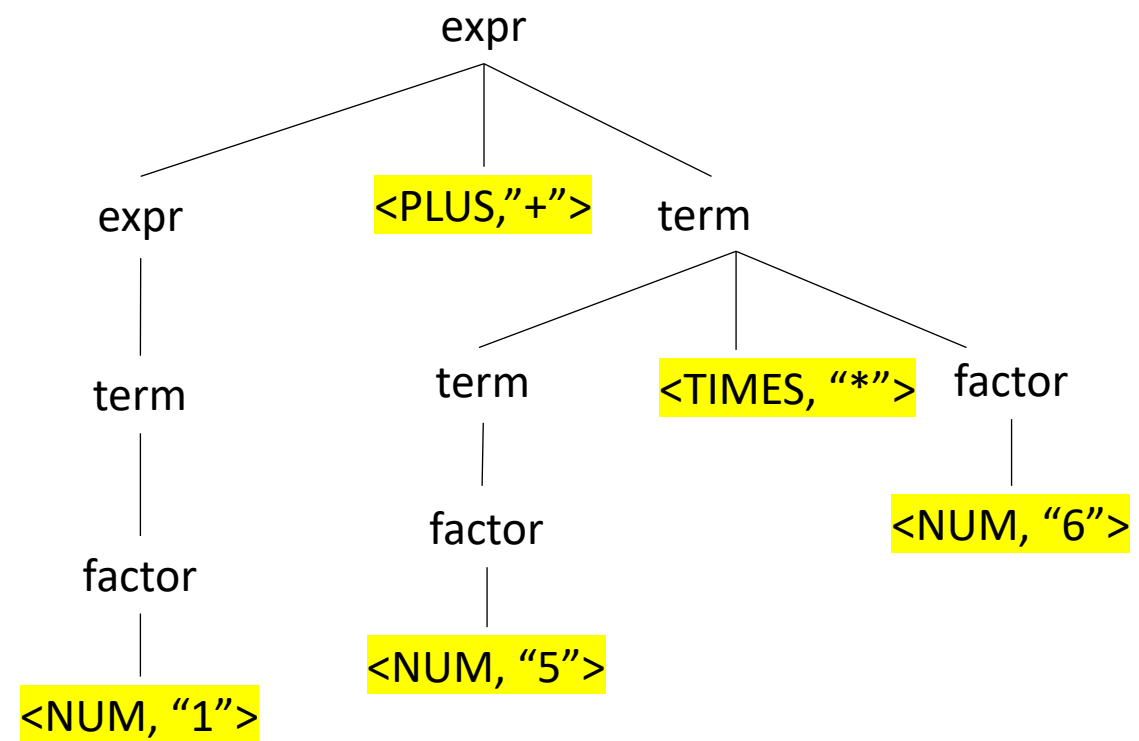


What is an AST?

input: 1+5*6

We'll start by looking at a parse tree:

Operator	Name	Productions
+	expr	: expr PLUS term term
*	term	: term TIMES factor factor
()	factor	: LPAREN expr RPAREN NUM



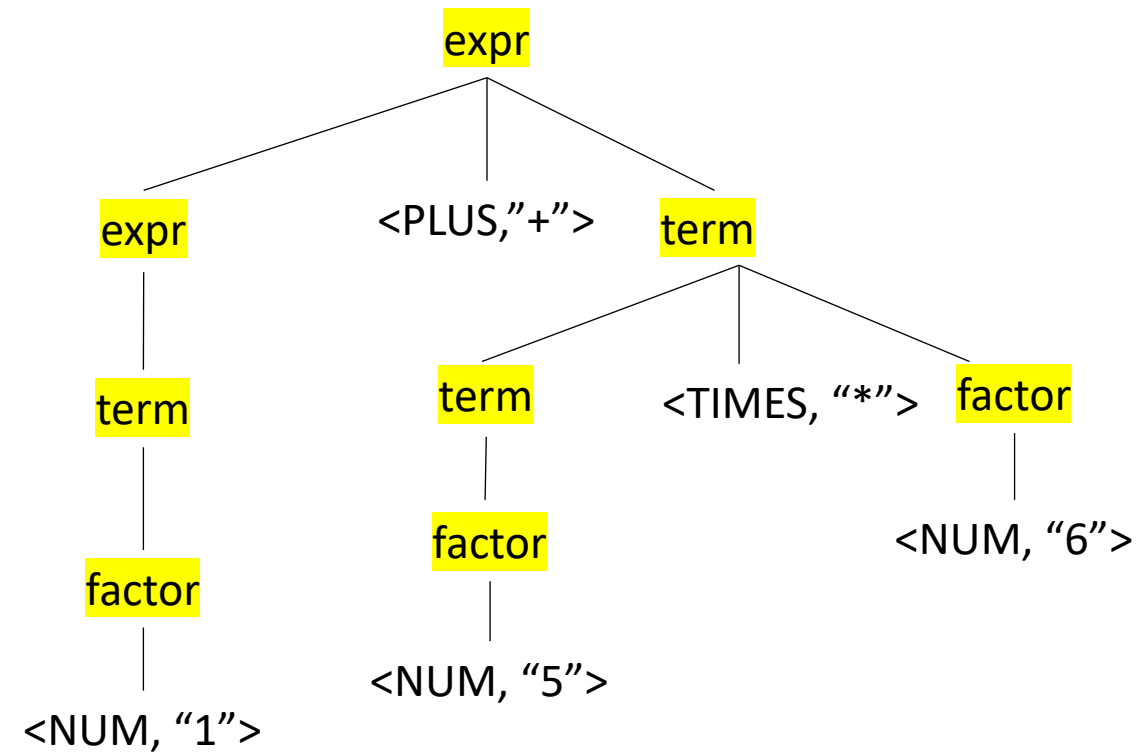
What are leaves? lexemes

What is an AST?

input: 1+5*6

We'll start by looking at a parse tree:

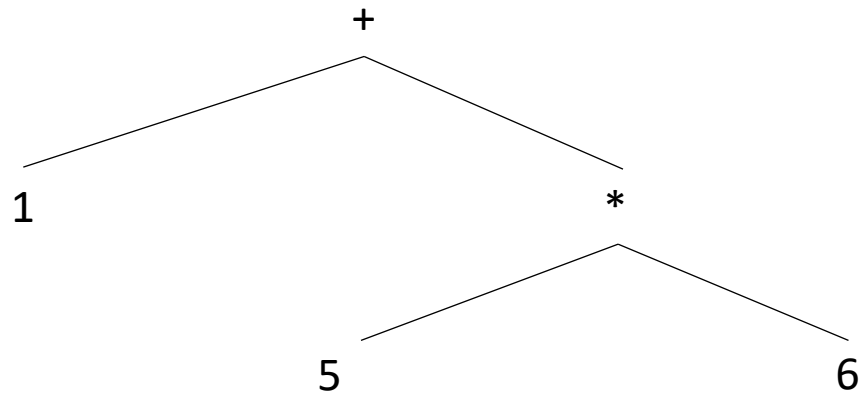
Operator	Name	Productions
+	expr	: expr PLUS term term
*	term	: term TIMES factor factor
()	factor	: LPAREN expr RPAREN NUM



What are nodes? non-terminals

What is an AST?

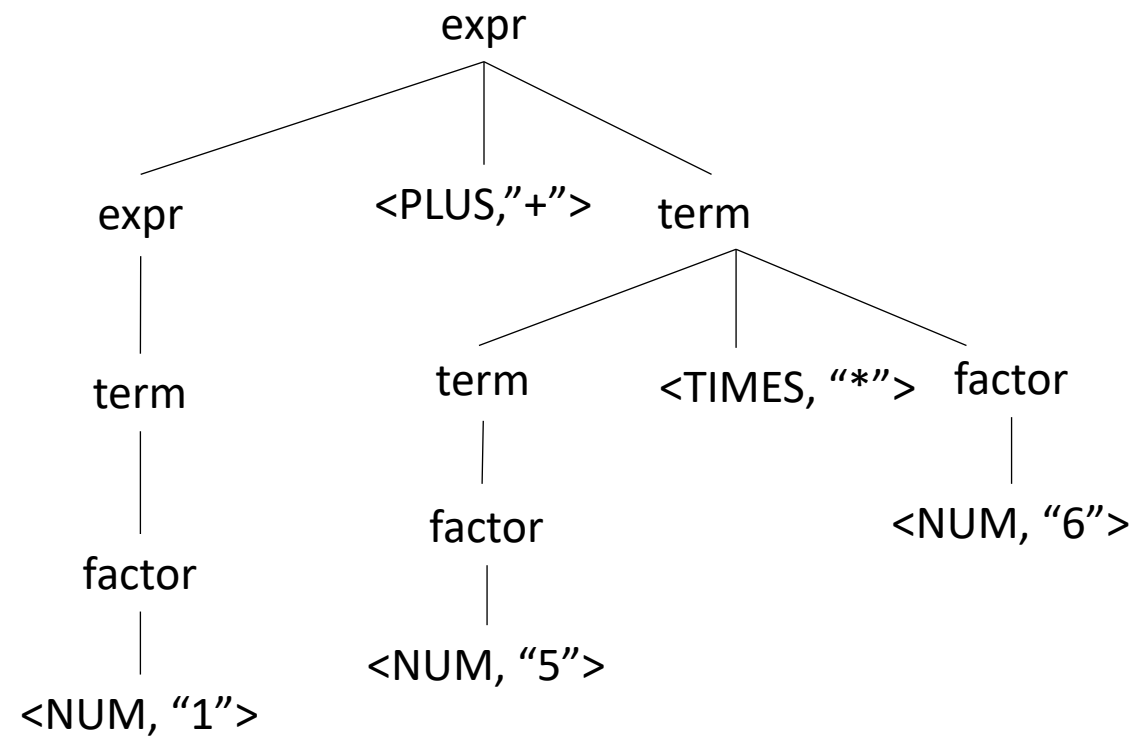
input: 1+5*6



AST

What are some differences?

- disjoint from the grammar
- leaves are data, not lexemes
- nodes are operators, not non-terminals

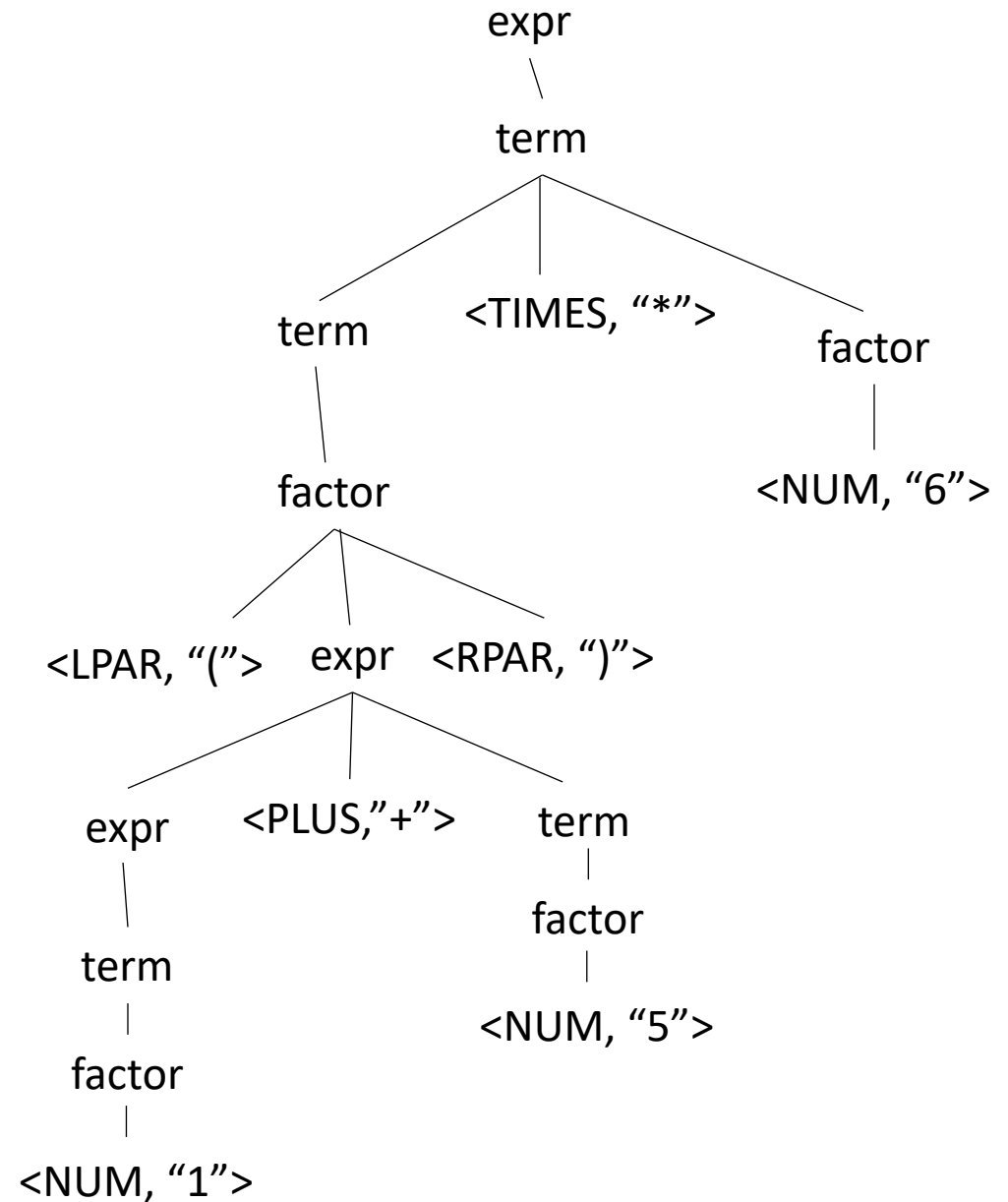


Example

what happens to ()s in an AST?

Operator	Name	Productions
+	expr	: expr PLUS term term
*	term	: term TIMES factor factor
()	factor	: LPAR expr RPAR NUM

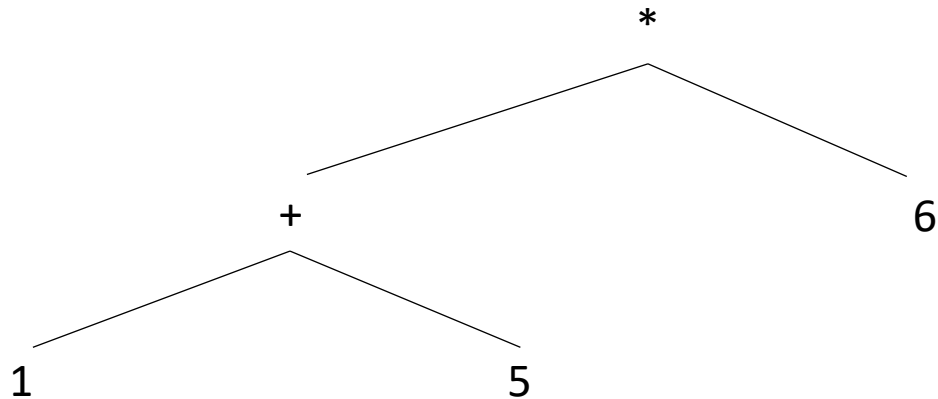
input: (1+5)*6



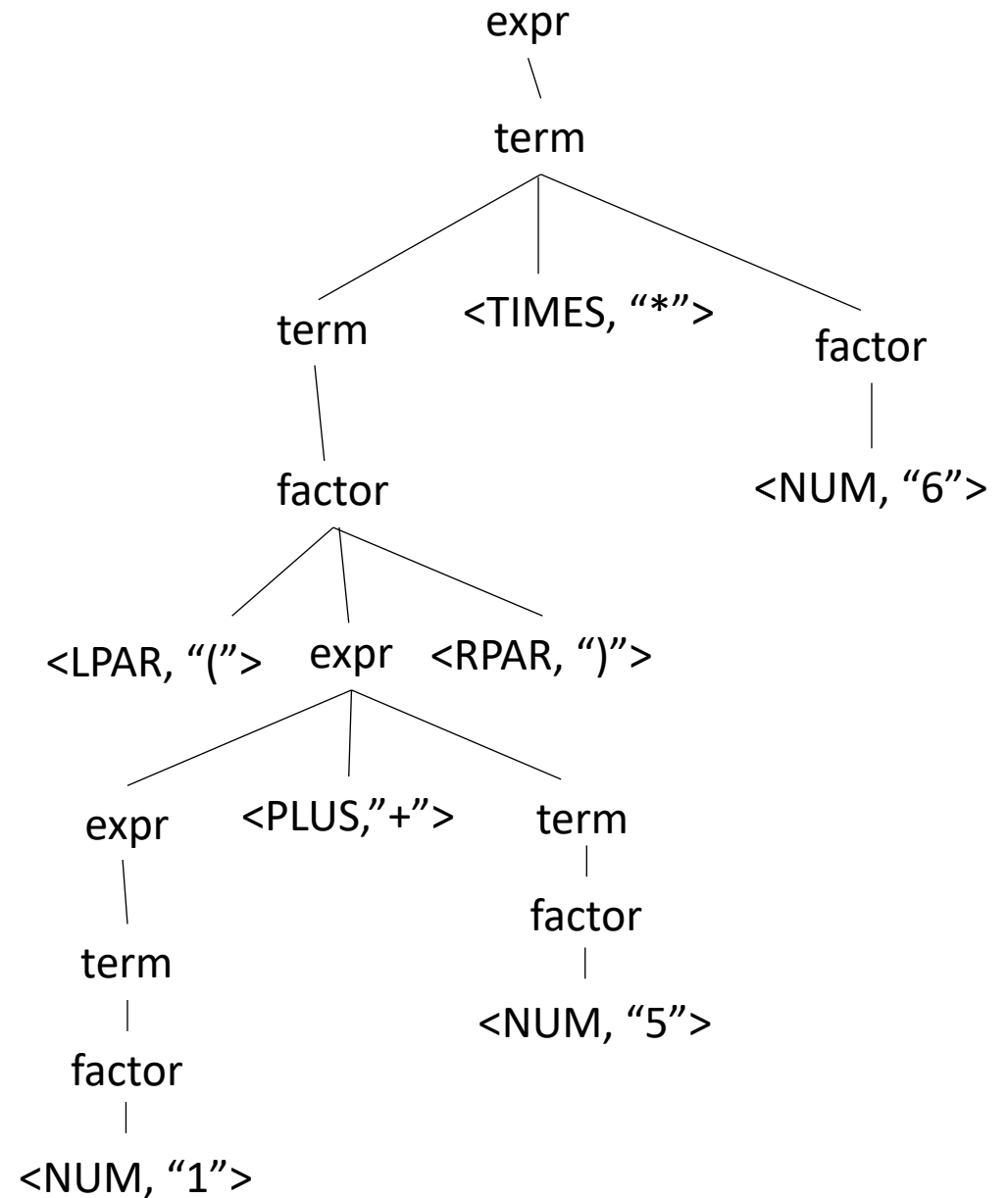
input: $(1+5) * 6$

Example

what happens to ()s in an AST?



No need for (), they simply encode precedence. And now we have precedence in the AST tree structure

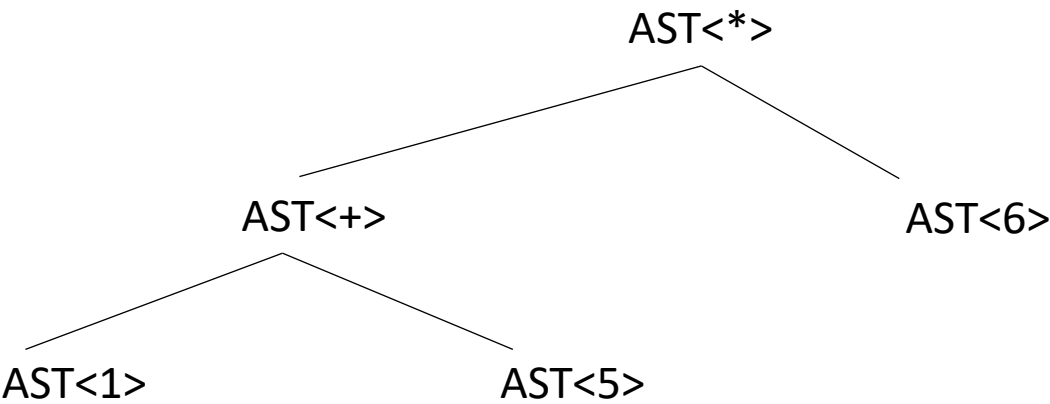
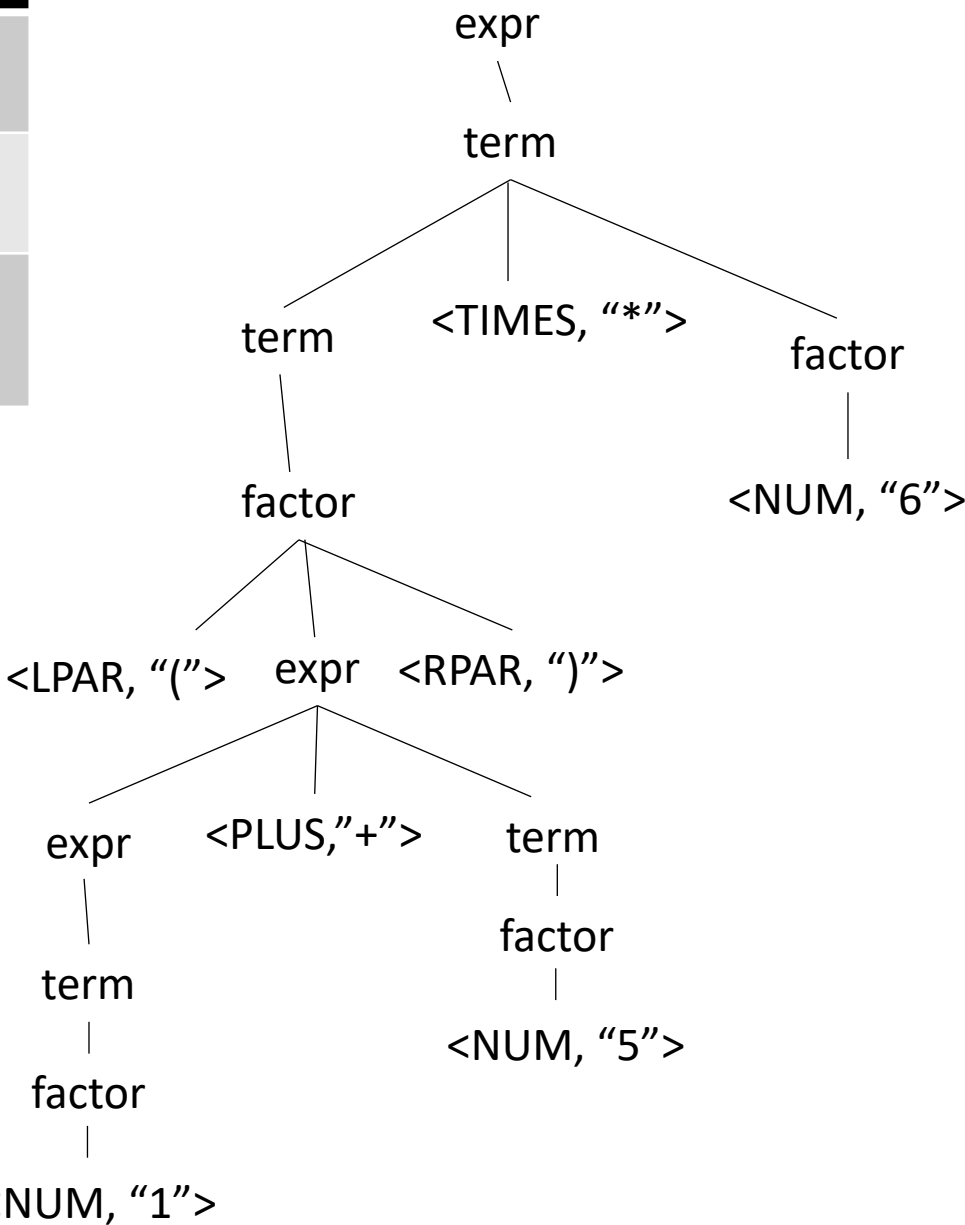


formalizing an AST

- A tree based data structure, used to represent expressions
- Main building block: Node
 - Leaf node: ID or Number
 - Node with one child: Unary operator (–) or type conversion (`int_to_float`)
 - Node with two children: Binary operator (+, *)

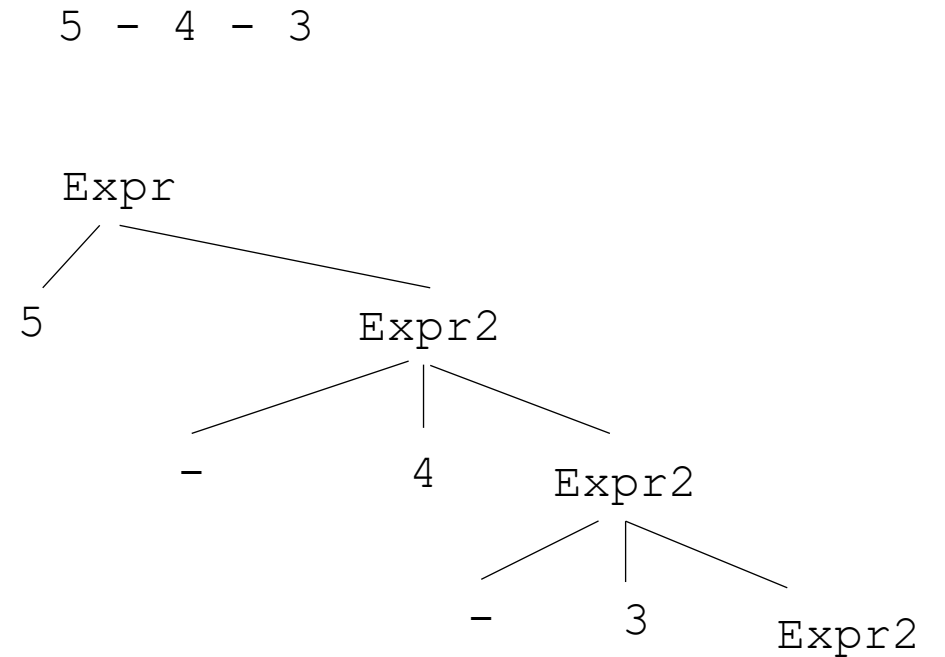
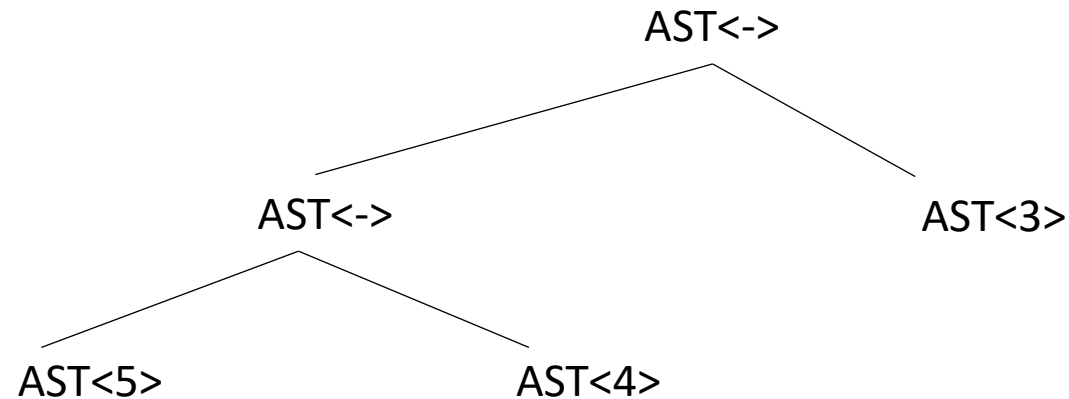
Name	Productions	Production action
expr	: expr PLUS term term	{return ASTAddNode(\$1,\$3)} {return \$1}
term	: term TIMES factor factor	{return ASTMultNode(\$1,\$3)} {return \$1}
factor	: LPAR expr RPAR NUM ID	{return \$2} {return ASTNumNode(\$1)} {return ASTIDNode(\$1)}

input: (1+5)*6



Creating an AST from top down grammar

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      | ""
```

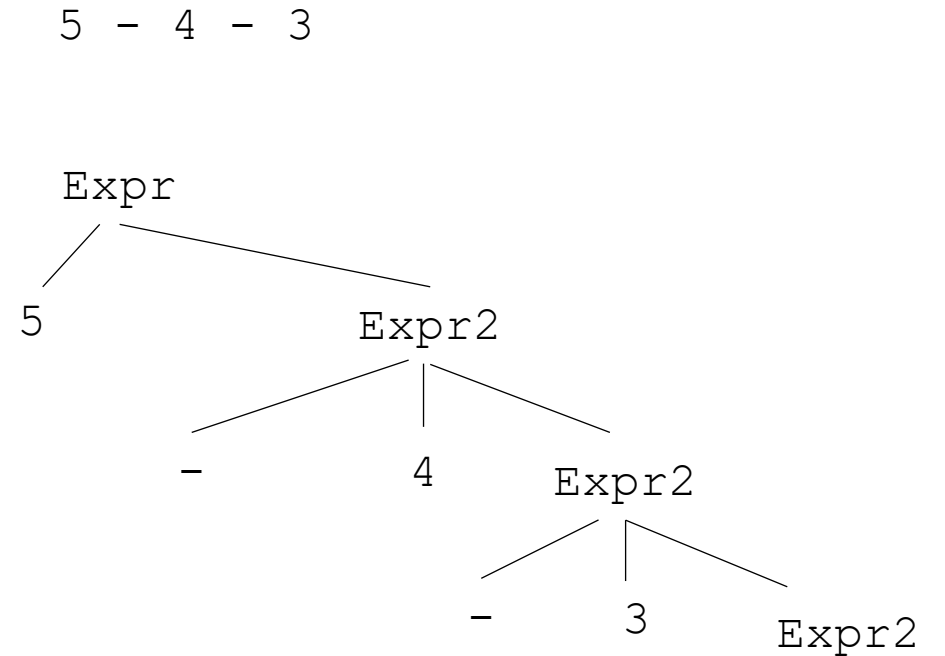


How do we get to the desired parse tree?

Creating an AST from top down grammar

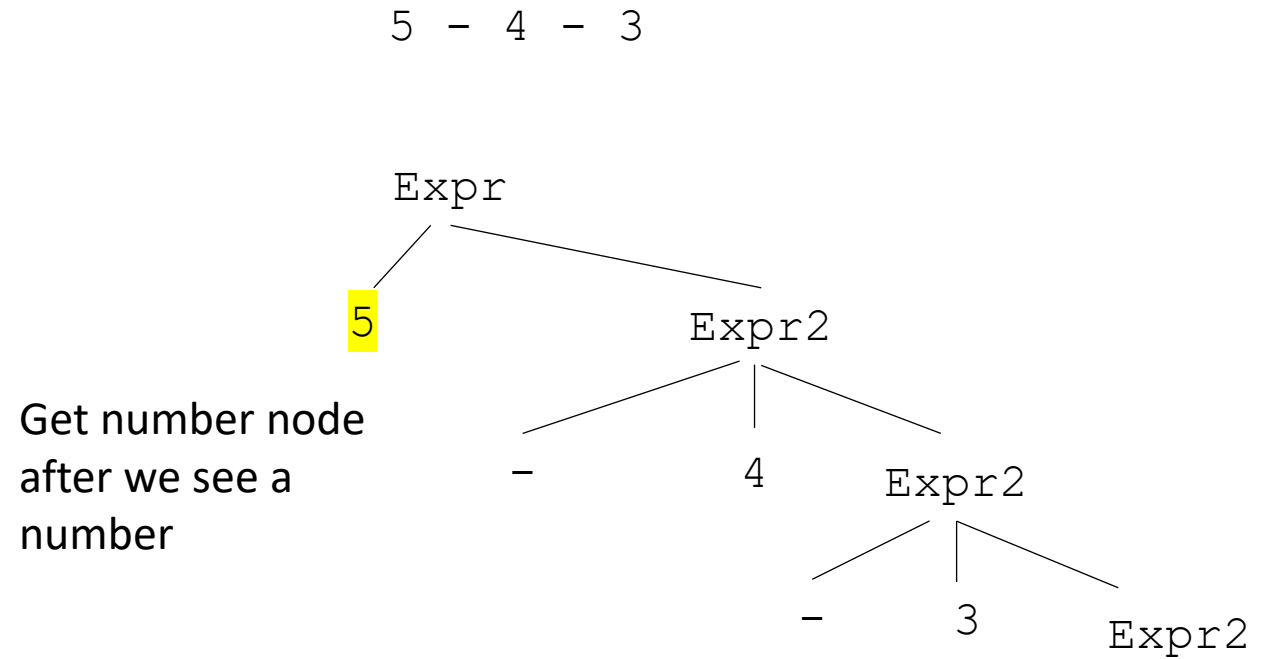
```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      | ""
```

Keep in mind that because we wrote our own parser, we can inject code at any point during the parse.



Creating an AST from top down grammar

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      | ""
```

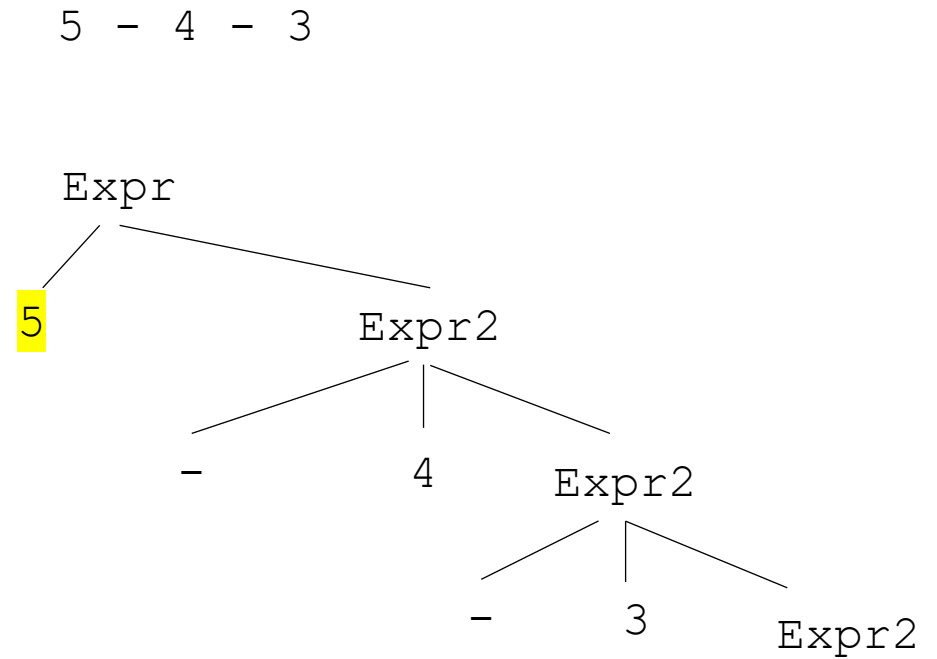


AST<5>

Creating an AST from top down grammar

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      | ""
```

Pass the node
down



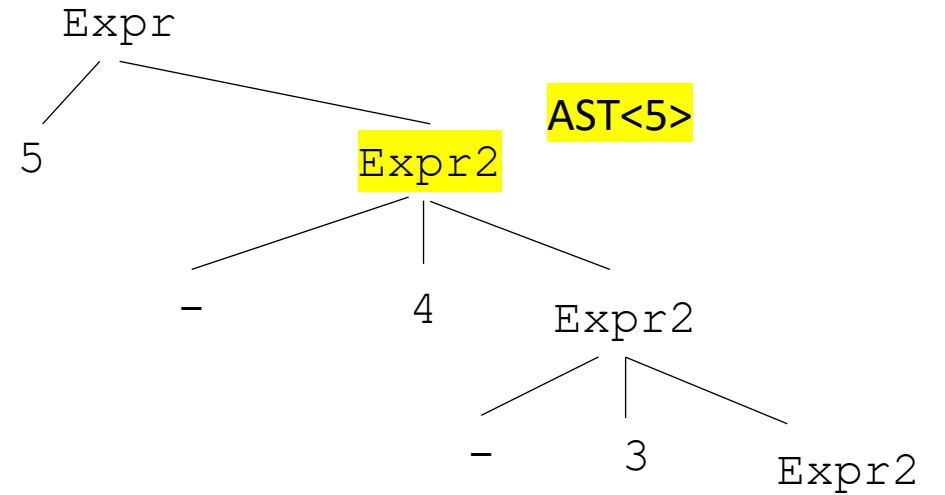
AST<5>

Creating an AST from top down grammar

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      | ""
```

5 - 4 - 3

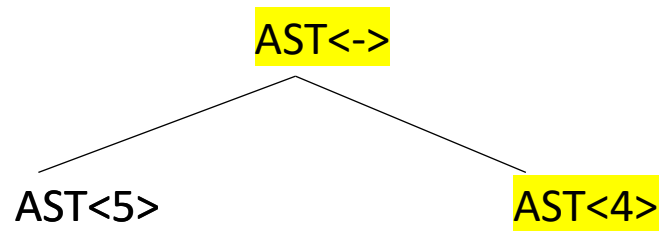
Pass the node
down



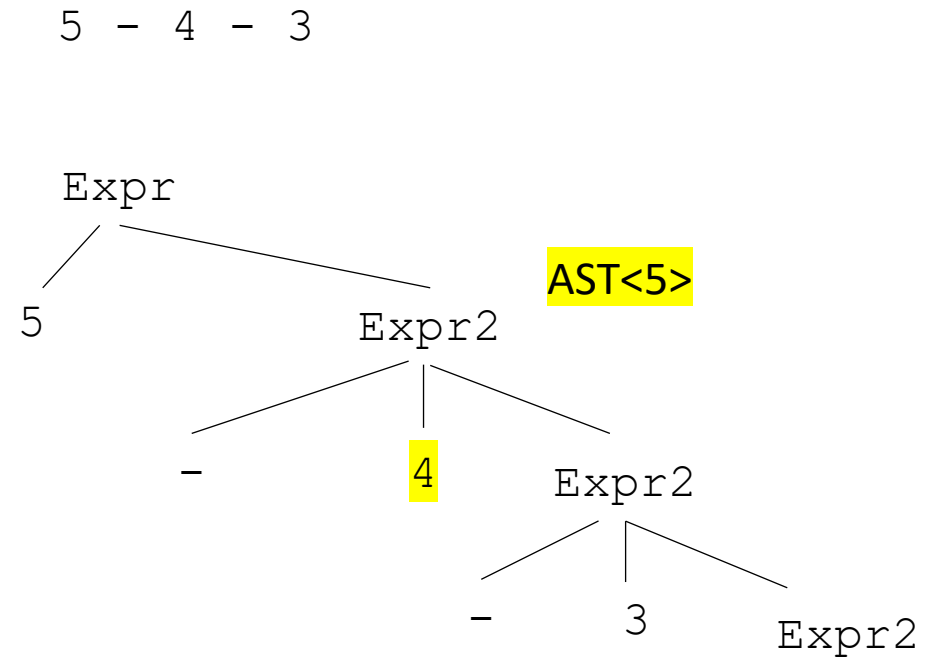
AST<5>

Creating an AST from top down grammar

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      | ""
```



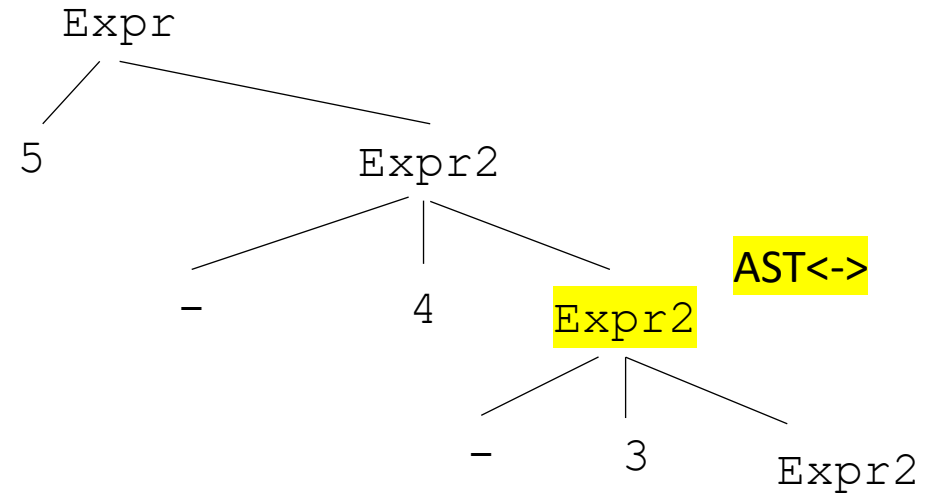
In Expr2, after 4 is
parsed, create a
number node and
a minus node



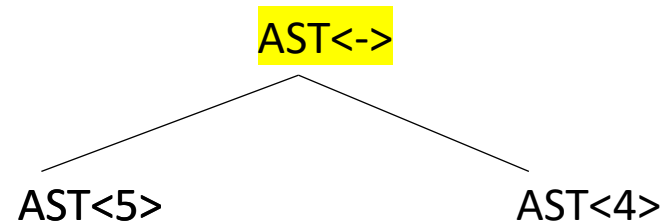
Creating an AST from top down grammar

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      | ""
```

5 - 4 - 3

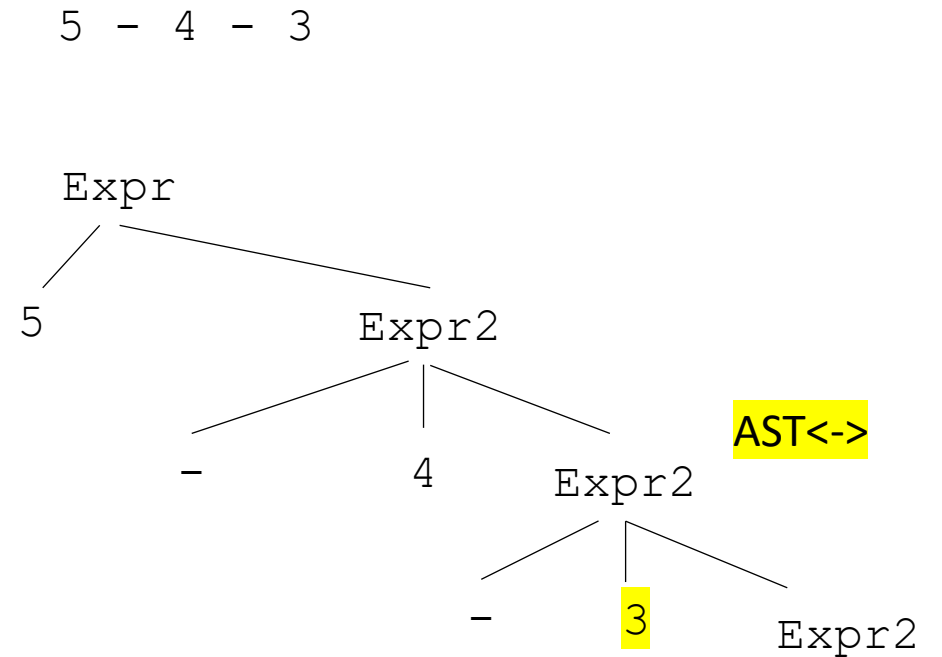
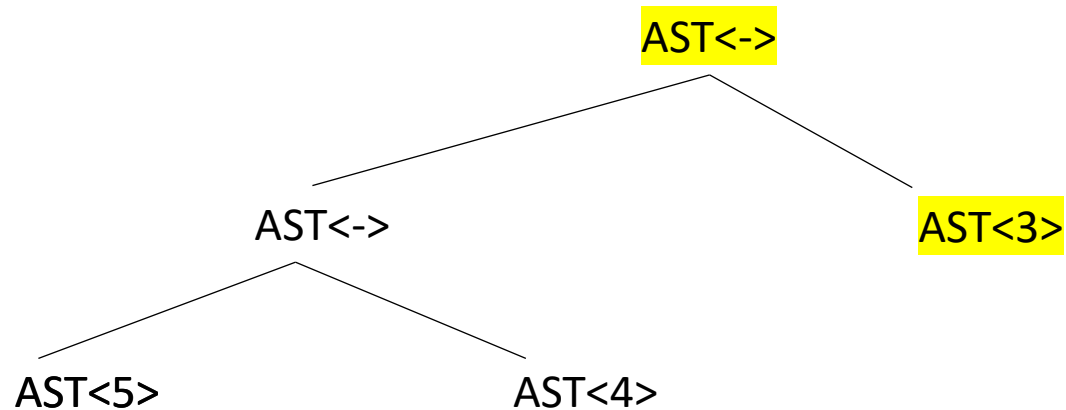


pass the new node
down



Creating an AST from top down grammar

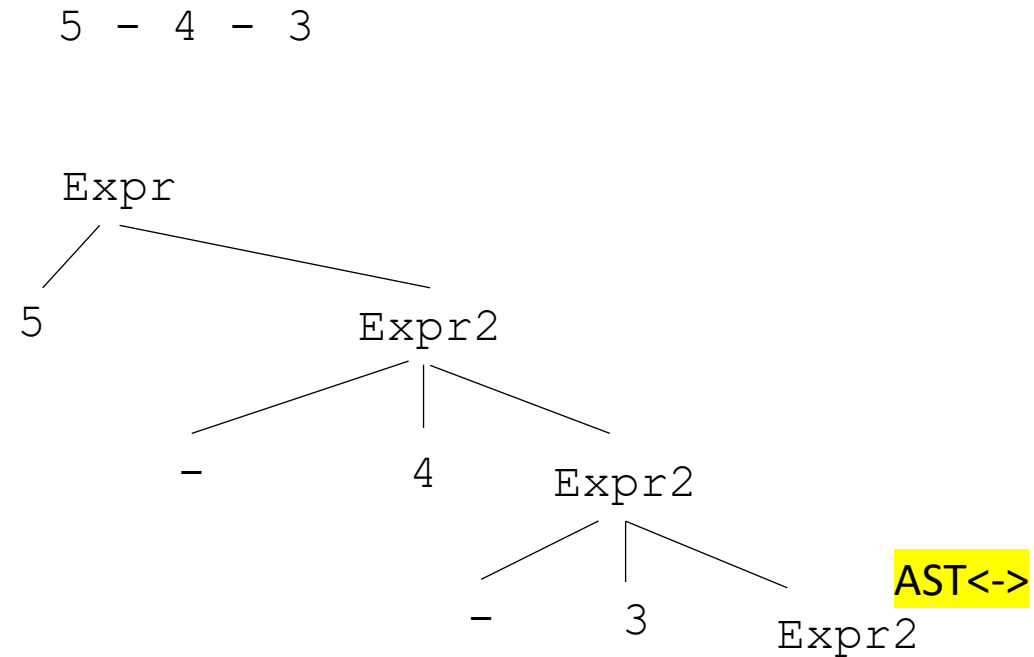
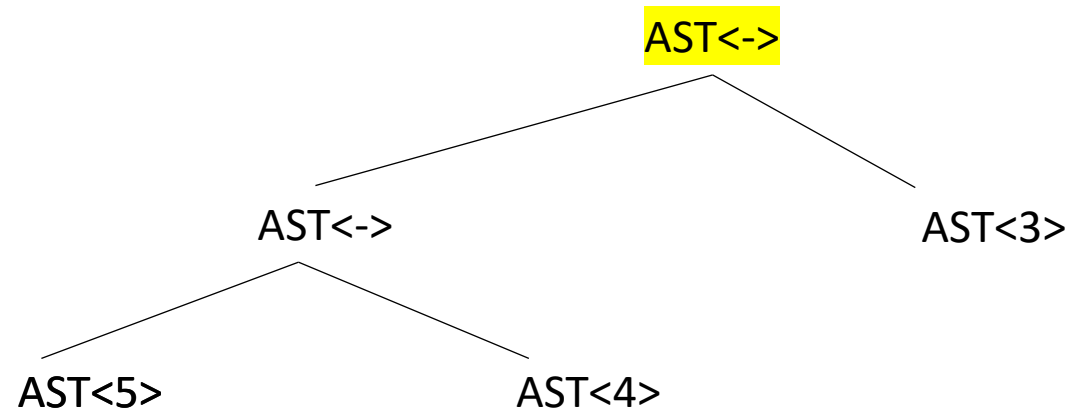
```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      | ""
```



In Expr2, after 3 is
parsed, create a
number node and
a minus node

Creating an AST from top down grammar

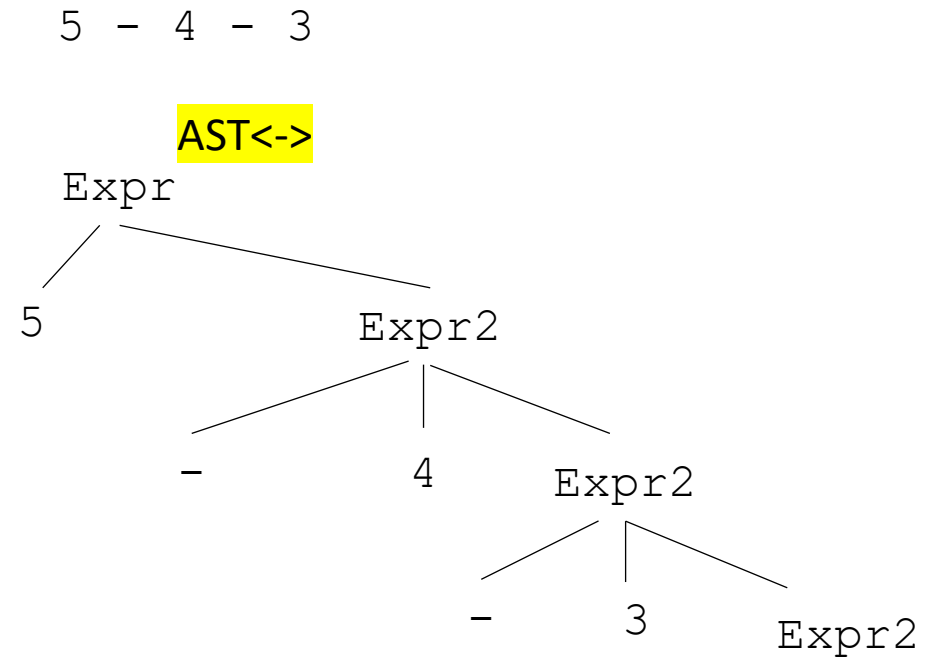
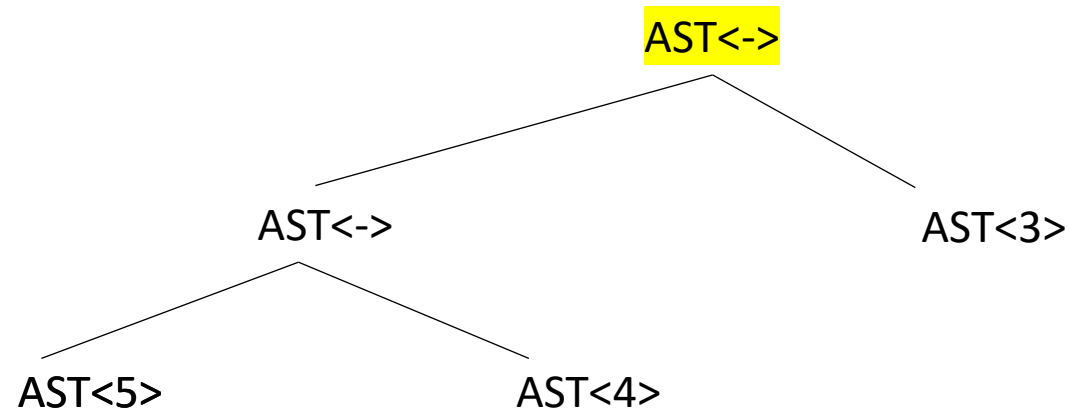
```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      | ""
```



pass down the new
node

Creating an AST from top down grammar

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      | ""
```



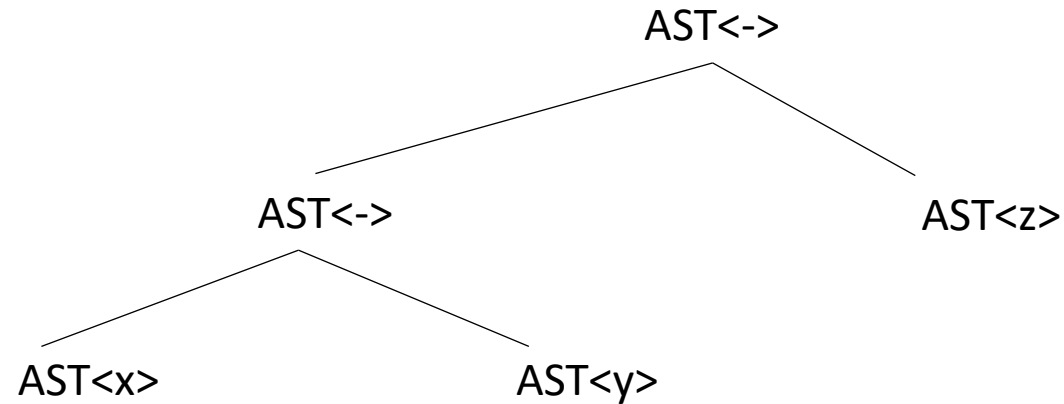
return the node
when there is
nothing left to
parse

Evaluate an AST by doing a post order traversal

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      |  ""
```

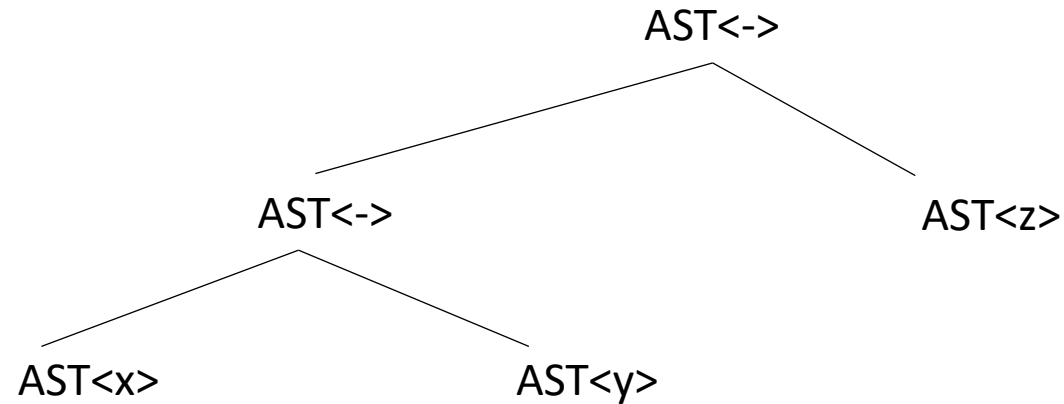
*What if you cannot evaluate it?
What else might you do?*

x - y - z



Evaluate an AST by doing a post order traversal

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      |  ""
```



What if you cannot evaluate it?

What else might you do?

```
int x;
int y;
float z;
float w;
w = x - y - z
```

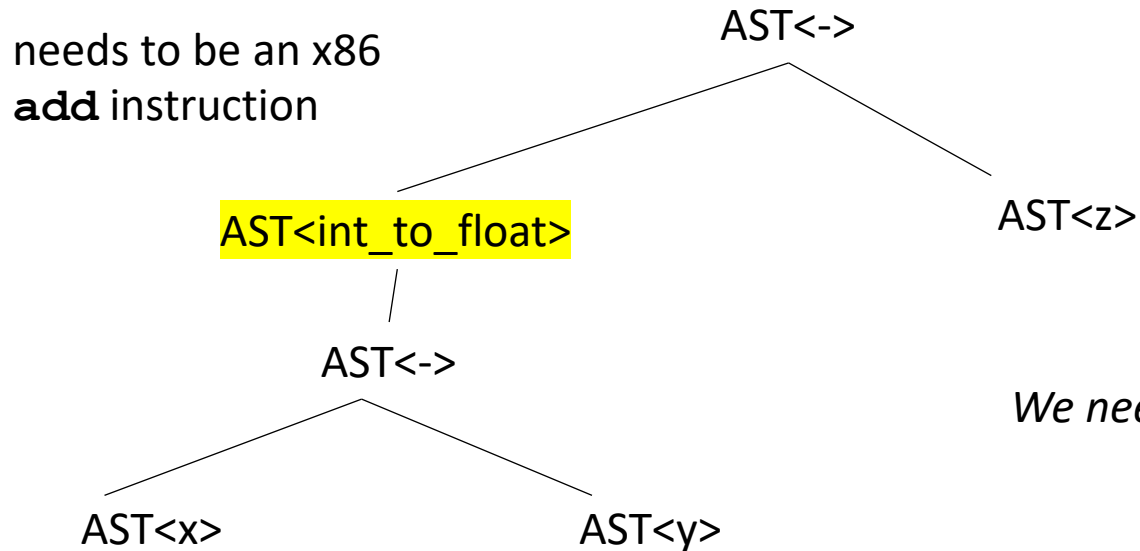
How does this change things?

Evaluate an AST by doing a post order traversal

```
Expr  ::= NUM Expr2
Expr2 ::= MINUS NUM Expr2
      |  ""
```

```
int x;
int y;
float z;
float w;
w = x - y - z
```

needs to be an x86
addss instruction



We need to make sure our operands are in the right format!

Type systems

- Given a language a type system defines:
 - The primitive (base) types in the language
 - How the types can be converted to other types
 - implicitly or explicitly
 - How the user can define new types

Type checking and inference

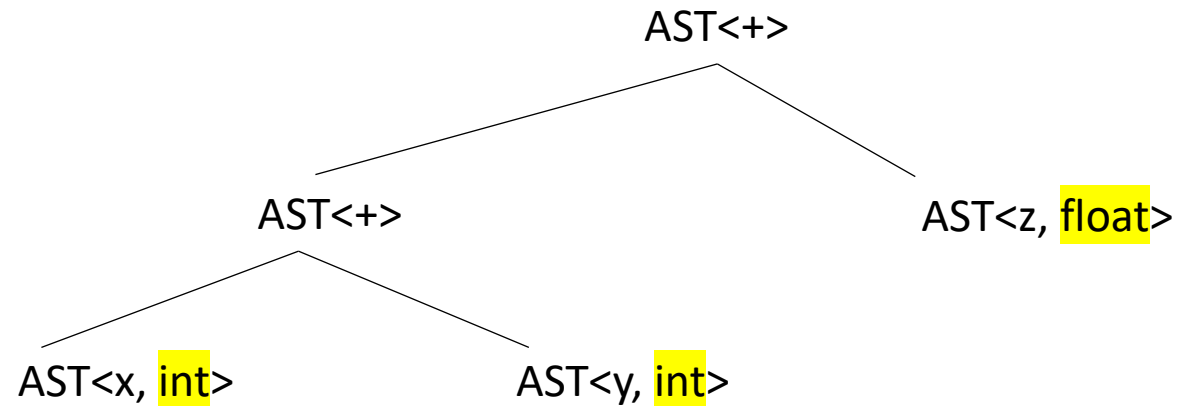
- Check a program to ensure that it adheres to the type system

Especially interesting for compilers as a program given in the type system for the input language must be translated to a type system for lower-level program

Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x + y + z
```

*each node additionally gets a type
we can get this from the symbol table for the leaves or based
on the input (e.g. 5 vs 5.0)*



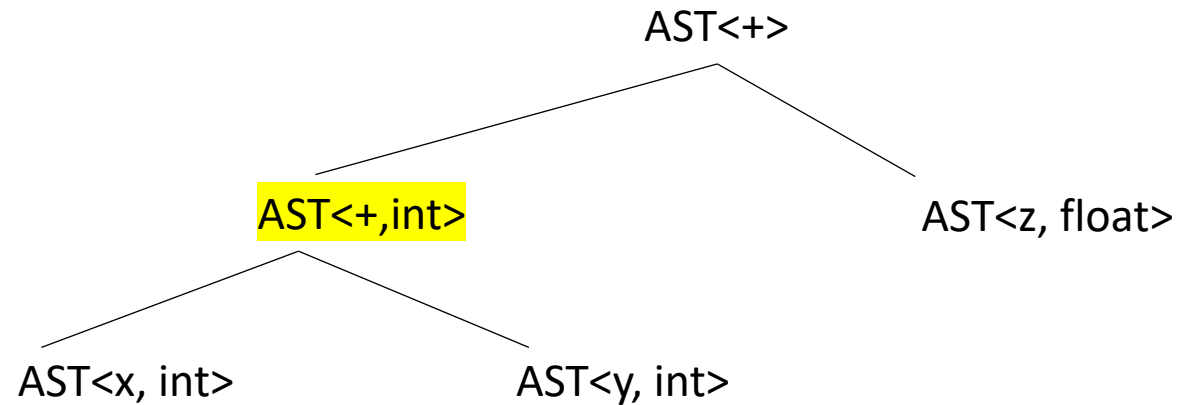
Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x + y + z
```

How do we get the type for this one?

inference rules for addition:

first	second	result
int	int	int
int	float	float
float	int	float
float	float	float



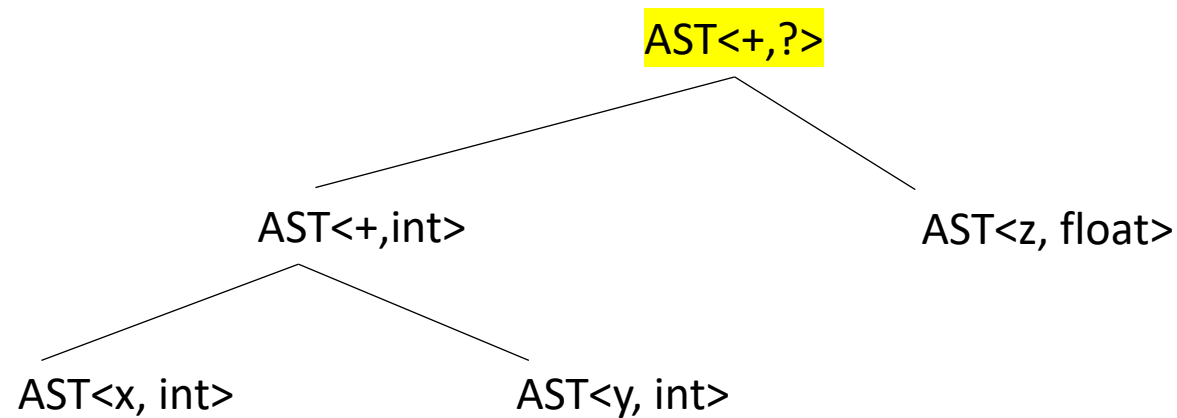
Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x + y + z
```

How do we get the type for this one?

inference rules for addition:

first	second	result
int	int	int
int	float	float
float	int	float
float	float	float



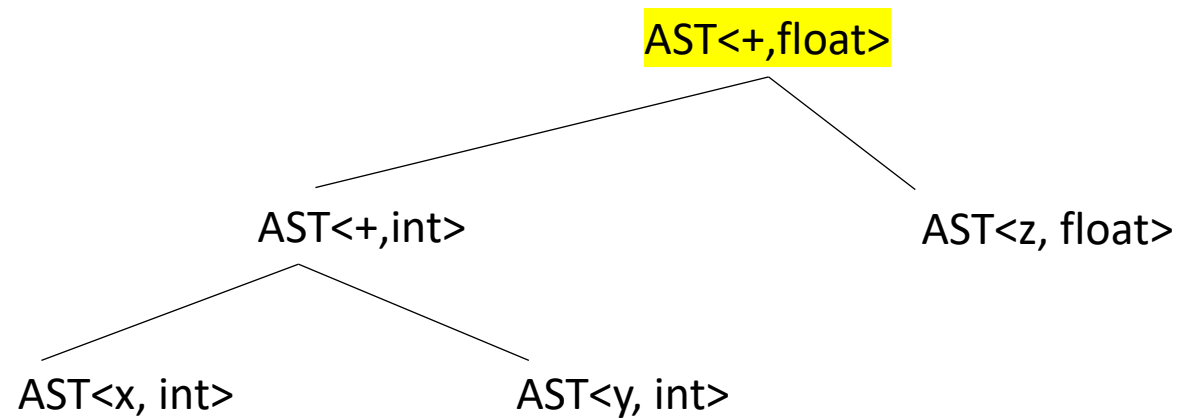
Type checking on an AST

```
int x;  
int y;  
float z;  
float w;  
w = x + y + z
```

How do we get the type for this one?

inference rules for addition:

first	second	result
int	int	int
int	float	float
float	int	float
float	float	float



Type checking on an AST

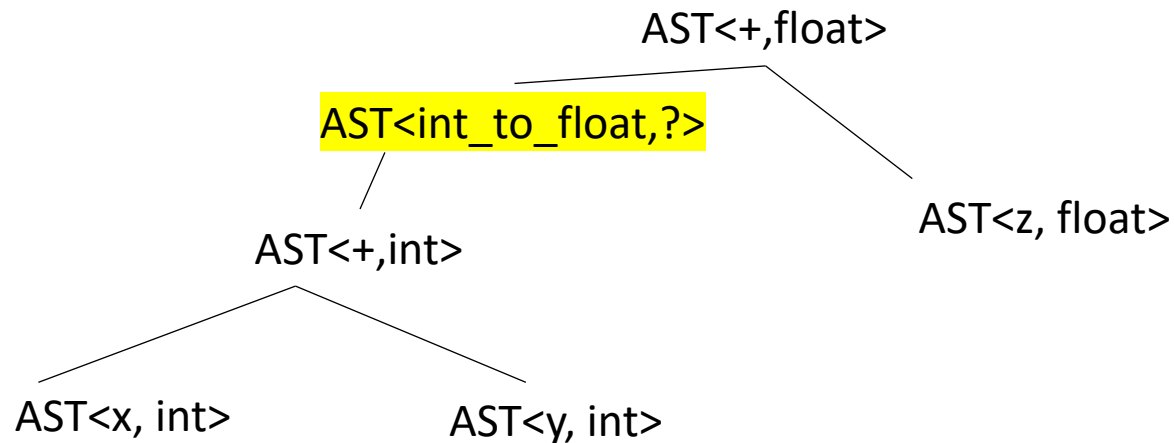
```
int x;  
int y;  
float z;  
float w;  
w = x + y + z
```

How do we get the type for this one?

inference rules for addition:

first	second	result
int	int	int
int	float	float
float	int	float
float	float	float

what else? need to convert the int to a float



Intermediate Representations

RULES FOR OUR IR

Class-IR: The Players

Inputs/outputs (IO): 32-bit typed inputs

e.g.: `int x, int y, float z` // e.g. params to a function

Program Variables (Variables): 32-bit untyped virtual register

given as `vrX` where X is an integer:

e.g. `vr0, vr1, vr2, vr3 ...`

Constants (float or ints): e.g. `3.5, 3e5, 6, 1024`

we will assume input/output names are disjoint from virtual register names

Class-IR

binary operators:

```
dst = operation(op0, op1);
```

operations can be one of these:

```
[add, sub, mult, div, eq, lt]
```

each operation is followed by an "i" or "f", which specifies how the bits in the registers are interpreted, eg **multi** for integers, **multf** for floating point.

Class-IR

binary operators:

```
dst = operation(op0, op1);
```

operations can be one of:

```
[add/i|f, sub/i|f, mult/i|f, div/i|f, eq/i|f, lt/i|f]
```

*all of **dst, op0, & op1** must be untyped virtual regs*

Class-IR: Examples

binary operators:

```
dst = operation(op0, op1);
```

Examples:

```
vr0 = addi(vr1, vr2);
```

```
vr3 = subf(vr4, vr5);
```

```
x = multf(vr0, vr1); not allowed!
```

```
vr0 = addi(vr1, 1); not allowed!
```

*We'll talk about how to
do this using other
instructions*

Class-IR: Control Flow

Control flow

`branch(label);`

- branches unconditionally to the label

`bne(op0, op1, label)`

- if op0 is not equal to op1 then branch to label
- operands must be virtual registers!

`beq(op0, op1, label)`

- Same as bne except it is for equal

Class-IR

Assignment

```
vr0 = vr1
```

one virtual register can be assigned to another

Class-IR

Assignment

```
vr0 = vr1
```

one virtual register can be assigned to another

Examples:

```
vr0 = 1; not allowed
```

```
vr1 = x; not allowed
```

Class-IR

unary get untyped register

```
dst = operation(op0);
```

operations are: [int2vr, float2vr]

Example:

Given IO: int x

```
vr1 = int2vr(x);
```

```
vr2 = float2vr(2.0);
```

Class-IR

unary get typed data for IO

```
dst = operation(op0);
```

operations are: [vr2int, vr2float]

Example:

Given IO: int x and float y

```
x = vr2int(vr1);  
y = vr2float(vr3);
```

Class-IR

unary conversion operators for VRs:

```
dst = operation(op0);
```

operations can be one of:

```
[vr_int2float, vr_float2int]
```

converts the bits in a **virtual register** from **one type to another**. ***op0** and **dst** must be a **virtual register!***

Class-IR: Examples

unary conversion operators:

```
dst = operation(op0);
```

Examples:

```
vr0 = vr_int2float(vr1);
```

```
vr2 = vr_float2int(1.0); not allowed!
```

Example

adding the values 1 - 9 to an input/output variable: int x

```
vr0 = int2vr(1);
```

```
vr1 = int2vr(1);
```

```
vr2 = int2vr(10);
```

```
loop_start:
```

```
vr3 = lti(vr0, vr2);
```

```
bne(vr3, vr1, end_label);
```

```
vr4 = int2vr(x);
```

```
vr5 = addi(vr4, vr0);
```

```
x = vr2int(vr5);
```

```
vr0 = addi(vr0, vr1);
```

```
branch(loop_start);
```

```
end_label:
```

Scopes

```
int x_0;  
int y_0;  
x_0 = 5;  
{  
  int x_1;  
  x = 6;  
  y = x;  
}
```

Let's walk through it with a symbol table

new scope. Add x with a new name

HT1

x: (INT, VAR, "x_1")

HT0

x: (INT, VAR, "x_0")
y: (INT, VAR, "y_0")

symbol table hash table stack

IR Program structure

- A sequence of 3 address instructions
- Programs can be split into **Basic Blocks**:
 - A **sequence of 3 address instructions** such that: there is a **single entry, single exit**
- *Important property*: an instruction in a basic block can assume that all preceding instructions will execute

How might they appear in a high-level language?

How many basic blocks?

```
...  
if (x) {  
    ...  
}  
else {  
    ...  
}  
...
```

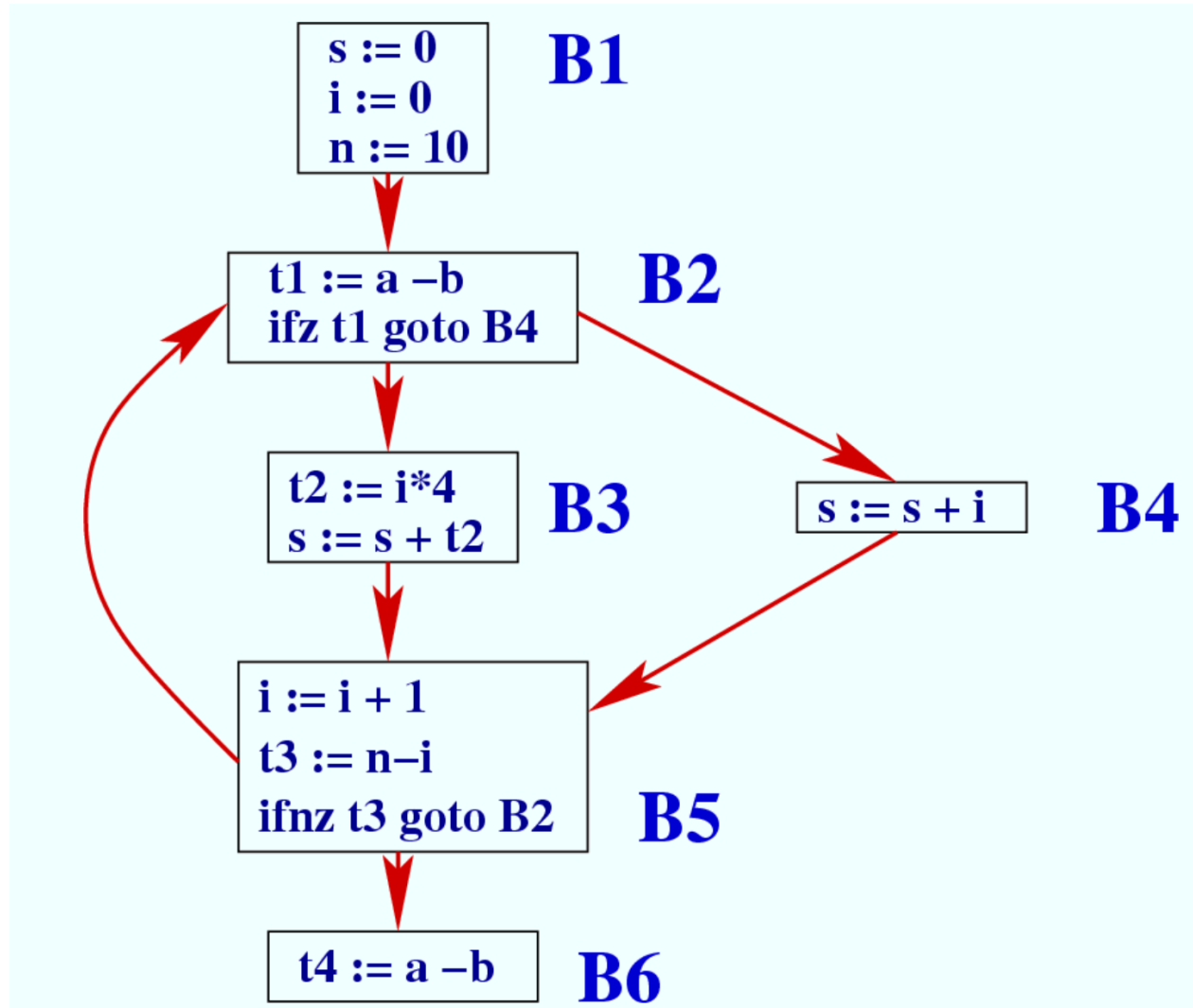
Two Basic Blocks

Single Basic Block

```
Label_x:  
op1;  
op2;  
op3;  
br label_z;
```

```
Label_x:  
op1;  
op2;  
op3;  
  
Label_y:  
op4;  
op5;
```

Example Control Flow Graph



How might they appear in a high-level language?

How many basic blocks?

```
...  
if (x) {  
    ...  
}  
else {  
    ...  
}  
...
```

Single Basic Block

```
Label_x:  
op1;  
op2;  
op3;  
br label_z;
```

Two Basic Blocks

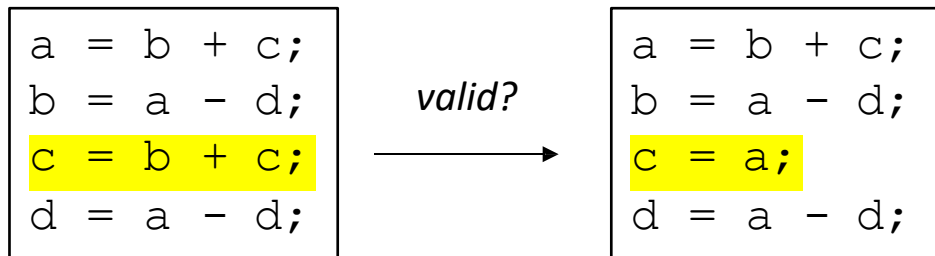
```
Label_x:  
op1;  
op2;  
op3;  
  
Label_y:  
op4;  
op5;
```

Local Value Numbering Optimization



Local value numbering

- A local optimization over 3 address code
- Attempts to replace arithmetic operations (expensive) with copy instructions (cheap)
- Can be extended to a regional optimization using flow analysis



Local value numbering

- A local optimization over 3 address code
- Attempts to replace arithmetic operations (expensive) with copy instructions (cheap)
- Can be extended to a regional optimization using flow analysis

```
a = b + c;  
b = a - d;  
c = b + c;  
d = a - d;
```

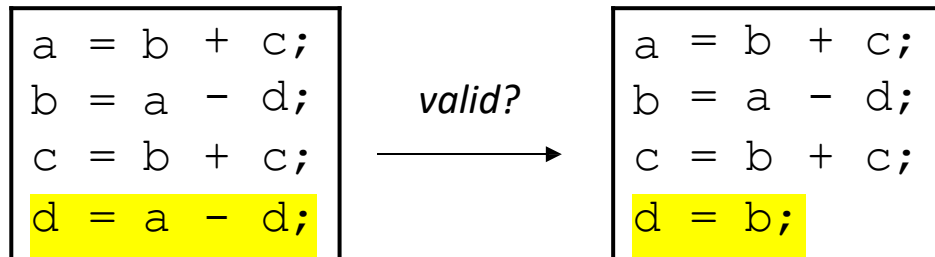
valid?

```
a = b + c;  
b = a - d;  
c = a;  
d = a - d;
```

No! Because b is redefined

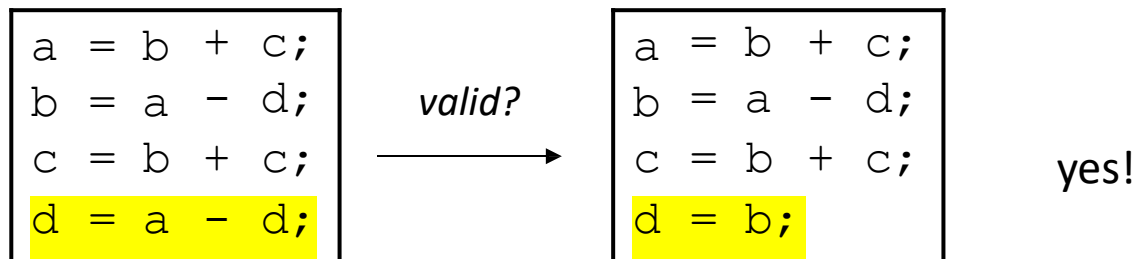
Local value numbering

- A local optimization over 3 address code
- Attempts to replace arithmetic operations (expensive) with copy instructions (cheap)
- Can be extended to a regional optimization using flow analysis



Local value numbering

- A local optimization over 3 address code
- Attempts to replace arithmetic operations (expensive) with copy instructions (cheap)
- Can be extended to a regional optimization using flow analysis



Local value numbering (LVN)

LVN Algorithm:

- Provide a number to each variable. Update the number each time the variable is updated.
- Keep a global counter; increment with new variables or assignments

a	=	b	+	c	;
b	=	a	-	d	;
c	=	b	+	c	;
d	=	a	-	d	;

Global_counter = 0

Local value numbering

Algorithm:

- Provide a number to each variable. Update the number each time the variable is updated.
- Keep a global counter; increment with new variables or assignments

a	=	b0	+	c1	;
b	=	a	-	d	;
c	=	b	+	c	;
d	=	a	-	d	;

Global_counter = 2

**b and c have not been
seen, give them a number**

Local value numbering

Algorithm:

- Provide a number to each variable. Update the number each time the variable is updated.
- Keep a global counter; increment with new variables or assignments

a2	=	b0	+	c1;
b	=	a2	-	d3;
c	=	b	+	c;
d	=	a	-	d;

Global_counter = 4

a2 have been updated use the last defined number. i.e. a on RHS, becomes a2 and so on.

Local value numbering

Algorithm:

- Provide a number to each variable. Update the number each time the variable is updated (assigned to).
- Keep a global counter; increment with new variables or assignments

a2	=	b0	+	c1;
b4	=	a2	-	d3;
c5	=	b4	+	c1;
d6	=	a2	-	d3;

Global_counter = 7

So ... for LHS variable always increment with updated number, for right hand side use, always appended last number version

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. **Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.**
- At each step, check to see if the rhs has already been computed.

a2	=	b0	+	c1;
b4	=	a2	-	d3;
c5	=	b4	+	c1;
d6	=	a2	-	d3;

If we remember to declare every one of these variable names, the IR is still valid !!!

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. **Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.**
- At each step, check to see if the rhs has already been computed.

→

a2	=	b0	+	c1;
b4	=	a2	-	d3;
c5	=	b4	+	c1;
d6	=	a2	-	d3;

H = {
}

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. **Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.**
- At each step, check to see if the rhs has already been computed.

→

a2	=	b0	+	c1;
b4	=	a2	-	d3;
c5	=	b4	+	c1;
d6	=	a2	-	d3;

H = {
 "b0 + c1" : "a2",
}

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.
- At each step, check to see if the rhs has already been computed.

→

a2	=	b0	+	c1;
b4	=	a2	-	d3;
c5	=	b4	+	c1;
d6	=	a2	-	d3;

H = {
 "b0 + c1" : "a2",
}

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. **Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.**
- At each step, check to see if the rhs has already been computed.

→

a2 = b0 + c1;
b4 = a2 - d3;
c5 = b4 + c1;
d6 = a2 - d3;

H = {
 "b0 + c1" : "a2",
 "a2 - d3" : "b4",
}

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.
- At each step, check to see if the rhs has already been computed.

→

a2	=	b0	+	c1;
b4	=	a2	-	d3;
c5	=	b4	+	c1;
d6	=	a2	-	d3;

H = {
 "b0 + c1" : "a2",
 "a2 - d3" : "b4",
}

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.
- At each step, check to see if the rhs has already been computed.

→

a2	=	b0	+	c1;
b4	=	a2	-	d3;
c5	=	b4	+	c1;
d6	=	a2	-	d3;

H = {
 "b0 + c1" : "a2",
 "a2 - d3" : "b4",
}

***mismatch due to
re-numbering!!!***

*i.e. it is no longer just
c = b + c*

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.
- At each step, check to see if the rhs has already been computed.

→

a2 = b0 + c1;
b4 = a2 - d3;
c5 = b4 + c1;
d6 = a2 - d3;

H = {
 "b0 + c1" : "a2",
 "a2 - d3" : "b4",
 "b4 + c1" : "c5",
}

Add new entry in
Hash table with new
Numbering.

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.
- At each step, check to see if the rhs has already been computed.

→

a2	=	b0	+	c1;
b4	=	a2	-	d3;
c5	=	b4	+	c1;
d6	=	a2	-	d3;

H = {

"b0 + c1" : "a2",

"a2 - d3" : "b4",

"b4 + c1" : "c5",

}

Do a look up and ...

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.
- At each step, check to see if the rhs has already been computed.

→

```
a2 = b0 + c1;  
b4 = a2 - d3;  
c5 = b4 + c1;  
d6 = b4;
```

```
H = {  
    "b0 + c1" : "a2",  
    "a2 - d3" : "b4",  
    "b4 + c1" : "c5",  
}
```

...match!

What else can we do?

Commutative operations

What is the definition of commutative?

$$x \text{ OP } y == y \text{ OP } x$$

What operators are commutative? Which ones are not?

Local value numbering: commutative operations

- For commutative operators (e.g. + *), deterministically re-order operands.
- You can use variable numbers or lexicographical order

→

a2	=	c1	-	b0;
f4	=	d3	*	a2;
c5	=	b0	-	c1;
d6	=	a2	*	d3;

H = {
}

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

cannot re-order because - is not commutative

→

a2	=	c1	-	b0;
f4	=	d3	*	a2;
c5	=	b0	-	c1;
d6	=	a2	*	d3;

H = {
 "c1 - b0" : "a2",
}

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

→

a2	=	c1	-	b0;
f4	=	d3	*	a2;
c5	=	b0	-	c1;
d6	=	a2	*	d3;

H = {
 "c1 - b0" : "a2",
}

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

→

a2	=	c1	-	b0;
f4	=	d3	*	a2;
c5	=	b0	-	c1;
d6	=	a2	*	d3;

re-ordered because a2 < d3 lexicographically

```
H = {  
    "c1 - b0" : "a2",  
    "a2 * d3" : "f4",  
}
```

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

→

a2	=	c1	-	b0;
f4	=	d3	*	a2;
c5	=	b0	-	c1;
d6	=	a2	*	d3;

```
H = {  
    "c1 - b0" : "a2",  
    "a2 * d3" : "f4",  
}
```

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

→

a2	=	c1	-	b0;
f4	=	d3	*	a2;
c5	=	b0	-	c1;
d6	=	a2	*	d3;

```
H = {  
    "c1 - b0" : "a2",  
    "a2 * d3" : "f4",  
    "b0 - c1" : "c5",  
}
```

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

→

a2	=	c1	-	b0;
f4	=	d3	*	a2;
c5	=	b0	-	c1;
d6	=	a2	*	d3;

H = {
 "c1 - b0" : "a2",
 "a2 * d3" : "f4",
 "b0 - c1" : "c5",
}

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

And achieves more optimization opportunities ...

→

a2 = c1 - b0;
f4 = d3 * a2;
c5 = b0 - c1;
d6 = f4;

```
H = {  
    "c1 - b0" : "a2",  
    "a2 * d3" : "f4",  
    "b0 - c1" : "c5",  
}
```

Stitching optimized code back into program

How to stitch optimized code back into program

Let's fix it

stitch part 1: *assign original
unnumbered variables their
latest values in each basic block*

```
a2 = b0 + c1;  
d5 = e3 + f4;  
g6 = b0 + c1;
```

```
a2 = b0 + c1;  
d5 = e3 + f4;  
g6 = a2;
```

```
label_0:  
h2 = g0 + a1;  
k3 = a1 + g0;
```

```
label_0:  
h2 = g0 + a1;  
k3 = h2;
```

```
a2 = b0 + c1;  
d5 = e3 + f4;  
g6 = a2;  
g = g6;  
d = d5;  
a = a2;
```

```
label_0:  
h2 = g0 + a1;  
k3 = h2;  
h = h2;  
k = k3;
```


How to stitch optimized code back into program

Method 1:

```
a2 = b0 + c1;  
d5 = e3 + f4;  
g6 = a2;  
g = g6;  
d = d5;  
a = a2;
```

```
label_0:  
h2 = g0 + a1;  
k3 = h2;  
h = h2;  
k = k3;
```

```
b0 = b  
c1 = c  
e3 = e  
f4 = f  
a2 = b0 + c1;  
d5 = e3 + f4;  
g6 = a2;  
g = g6;  
d = d5;  
a = a2;
```

Continue onto label_0
of other basic block

stitch part 2:

**Copy unnumbered inputs
to initial RHS numbered inputs**

```
label_0:  
g0 = g  
a1 = a  
h2 = g0 + a1;  
k3 = h2;  
h = h2;  
k = k3;
```

How to stitch optimized code back into program

Method 2: stitch part 2: **drop numbers from first use of RHS variables**

```
a2 = b0 + c1;  
d5 = e3 + f4;  
g6 = a2;  
g = g6;  
d = d5;  
a = a2;
```

```
a2 = b + c;  
d5 = e + f;  
g6 = a2;  
g = g6;  
d = d5;  
a = a2;
```

```
label_0:  
h2 = g0 + a1;  
k3 = h2;  
h = h2;  
k = k3;
```

```
label_0:  
h2 = g + a;  
k3 = h2;  
h = h2;  
k = k3;
```

How to stitch optimized code back into program

Now the Basic Blocks can be re-combined!!

```
a2 = b0 + c1;  
d5 = e3 + f4;  
g6 = a2;  
g = g6;  
d = d5;  
a = a2;
```

```
a2 = b + c;  
d5 = e + f;  
g6 = a2;  
g = g6;  
d = d5;  
a = a2;
```

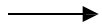
```
a2 = b + c;  
d5 = e + f;  
g6 = a2;  
g = g6;  
d = d5;  
a = a2;
```

```
label_0:  
h2 = g0 + a1;  
k3 = h2;  
h = h2;  
k = k3;
```

```
label_0:  
h2 = g + a;  
k3 = h2;  
h = h2;  
k = k3;
```

```
label_0:  
h2 = g + a;  
k3 = h2;  
h = h2;  
k = k3;
```

Basic and Generalized Loop Unrolling



Simpler Loop Unrolling Conditions

- Under what conditions can we unroll?

FOR LPAR **assignment_statement** **expr** SEMI **assignment_statement** RPAR **statement**

```
for (int i = 0; i < 128; i++) {  
    // body  
}
```

Validate that we actually have an iteration variable

1. **find** candidate on lhs of **assignment_statement**
2. **check** no assignments to candidate in **body**
3. **check** that it matches lhs of **assignment_statement**
4. **check** **loop condition**
 - * check that candidate variable is on lhs
 - * check that the **rhs is a literal**
(i.e. a **compile time constant value**, e.g. $i < 10 * 12$)

*Do these guarantee we will find an iteration variable?
What happens if we don't find one?*

Generalized Loop Unrolling Conditions

What about in the general case? For unroll factor F ?

```
for (int i = x; i < y; i++) {  
    // body  
}
```

find out how many unrolled loops we can execute:
?

This gives us the first bound

second loop is initialized with the first bound

second loop's bound is same as the original loop

what if we executed the unrolled loop as many times as it was valid, and did the rest with a non-unrolled loop

```
for (int i = x; i < y/F * F; i++) {  
    // body  
    i++  
    ...  
}
```

Note that $y/F * F$ creates a remainder
i.e. the benefit of integer arithmetic

```
for (int i = y/F * F; i < y; i++) {  
    // body  
}
```

Generalized Loop Unrolling Conditions

- general unroll

For unroll factor F

General unroll constraints:

- Loop update increments by 1
- Find the concrete number of loop iterations, LI

General unroll code generation:

- Create simple unrolled loop with new bound: $(LI/F)*F$
- Create cleanup (basic) loop with initialization: $(LI/F)*F$
- perform codegen

None of these numbers have to be concrete!